

Live NBA Win Probability Prediction Dashboard with Interactive Scenario Simulation

Tim Damasco, Brandon Barber, and Mohammad Islam

Abstract—Live win probability prediction has become increasingly valuable in sports analytics, yet existing systems provide limited interactivity and fail to incorporate comprehensive contextual features that influence game outcomes. Our project aimed to develop a machine learning model for a sports analytics dashboard that provides continuously updated win probabilities during live NBA games. It also introduces a what-if simulator, a novel tool that allows users to explore hypothetical scenarios, such as alternative lineups or scoring sequences, and observe their impact on win probability predictions. Our system integrates multiple data sources including play-by-play event data, team Elo ratings, dynamic player statistics, and real time lineup compositions to capture both pregame team strength and in game momentum shifts. Our model uses a stacking technique with Logistic Regression, XGBoost, and Random Forest which allows for the examination of both linear and non-linear relationships between features.

I. INTRODUCTION

THE purpose of this project is to create a data science solution that enhances the way live sports are experienced by both fans and analysts. Current trackers provide raw statistics and static win probability curves, but lack interactivity and predictive depth. For instance, ESPN’s Basketball Power Index (BPI) [2] offers pregame win probabilities but does not update dynamically based on in game events or allow users to explore hypothetical scenarios. Similarly, FiveThirtyEight’s NBA predictions [3] focus primarily on season-long forecasting and pregame probabilities rather than real time, play-by-play probability updates. While Inpredictable [6] has developed a functional in game win probability calculator for the NBA, it remains a passive tool that displays probabilities without enabling user interaction or scenario exploration. These existing systems, though valuable for providing baseline win expectations, do not offer the participatory element of simulating alternative game events or explaining probability shifts in accessible terms, leaving a large gap in how fans and analysts can engage with predictive analytics during live games.

Our dashboard fills this gap by offering continuously updated winning probabilities, presented through intuitive visualizations. The system updates probabilities as plays occur, incorporating dynamic lineup information and contextual game features that existing systems overlook. By tracking active player combinations and their cumulative season performance metrics, our approach captures substitution impacts and lineup quality in ways that static pregame models cannot. This becomes especially important with the implementation of our what-if simulator.

A. Beneficiaries and Applications

Beneficiaries of this system include:

- **Fans**, who gain engaging, live insight into how likely their team is to win and can explore “what-if” scenarios to understand the impact of coaching decisions and game events.
- **Sports media and commentators**, who can enhance live broadcasts with predictive analytics and natural language explanations that make complex statistics accessible to general audiences.
- **Teams and analysts**, who can benchmark in game decisions against probability forecasts and use the simulator to evaluate strategic alternatives.

B. Novel Contribution

The novel aspect of our work is the interactive “what-if” simulator, which enables users to adjust game factors and immediately see projected changes in win probability. This adds a participatory layer that does not exist in existing win probability dashboards. Users can test scenarios such as:

- Substituting specific players and observing how lineup quality impacts win probability
- Simulating scoring sequences (e.g., “What if the home team scores on the next three possessions?”)
- Fast forwarding time while holding score constant to understand time dependent dynamics

C. Anticipated Technical Challenges

Several technical challenges were anticipated and addressed:

- 1) real time data ingestion from NBA APIs with sufficient reliability and update frequency
- 2) Synchronizing and cleaning play-by-play data streams that contain inconsistencies, overlapping events, and missing values
- 3) Training predictive models that can update probabilities with minimal latency
- 4) Scaling the system to handle multiple games simultaneously without degrading performance
- 5) Integrating the what-if module without disrupting live updates
- 6) Feature engineering for dynamic lineup tracking across substitutions with inconsistent player naming conventions
- 7) Probability calibration to ensure predicted probabilities accurately reflect true outcome frequencies

II. RELATED WORK

A. General Machine Learning Framework for Sports Prediction

Bunker and Thabtah [7] establish a comprehensive framework for applying machine learning to sport result prediction, with particular emphasis on Artificial Neural Networks and their effectiveness in handling the non-linear relationships present in sports data. Their analysis identifies three critical data categories for successful predictive models: historical match outcomes, player performance metrics, and opposition characteristics. The authors highlight specific challenges in sports prediction that distinguish it from conventional classification tasks, including high dimensional feature spaces, class imbalance in outcome distributions, and the inherent dependencies between sequential game events.

Their proposed framework addresses model evaluation through cross-validation strategies that account for sequential ordering, preventing data leakage from future information into training sets. We adopt this validation approach in our work, ensuring that training data always precedes test data chronologically and that player statistics are calculated cumulatively only up to each game date.

B. NBA Win Probability Implementation

Unpredictable [6] presents an NBA win probability calculator utilizing locally weighted logistic regression, an algorithm that assigns greater weight to training examples similar to the current game state. The model employs four input variables: game time remaining, current point differential, possession status, and Vegas point spread. This feature selection demonstrates the tradeoff between bias and variance in sports prediction modeling, where a minimal feature set reduces model complexity and overfitting risk while capturing informative game state characteristics.

The inclusion of Vegas point spread is particularly noteworthy as it incorporates the collective predictions of betting markets as a pregame team strength indicator. This approach aligns with the principle of using external signals to enhance model predictions, similar to how we integrate Elo ratings in our modeling. Our work extends beyond this approach by incorporating dynamic player lineup information, thus enabling the model to capture the significant impact of substitutions on win probability.

C. Soccer Win Probability Modeling

Robberechts [5] developed an in game win probability model for soccer using machine learning techniques that emphasize feature engineering and sequential modeling. Their work addresses the challenge of capturing momentum and contextual game dynamics through time-series features and statistics that encode recent performance trends. The model uses gradient boosting methods, which are effective for sports prediction due to their ability to model complex, nonlinear interactions between features while maintaining interpretability through feature importance metrics.

This technique demonstrates the value of ensemble learning approaches that combine multiple weak learners to achieve

robust predictions. Their framework also highlights the importance of probability calibration, ensuring that predicted probabilities accurately reflect true outcome frequencies. We adopt both principles in our work: employing ensemble methods (specifically stacking) to combine logistic regression, random forest, and XGBoost, and rigorously evaluating calibration through Expected Calibration Error metrics. Our analysis confirms that ensemble approaches provide marginal but consistent improvements in both accuracy and calibration over individual models.

III. DATASET AND DATA COLLECTION

A. Data Sources

Our system integrates multiple data sources: the NBA API Python library [15] and FiveThirtyEight's historical Elo Ratings, to create a comprehensive feature set for win probability prediction. The data comes from multiple subpackages within the Python NBA API library.

- **PlayByPlayV3** - A comprehensive dataset from the `nba_api` package containing detailed play-by-play logs for every game of the NBA season. This dataset serves as the foundation for predicting win probability in our models. Key features include unique game identifiers, time stamped score updates, team information, and play descriptions that capture each significant event during the game. We collected play-by-play data for 1,750 games spanning multiple NBA seasons, resulting in over 1,000,000 individual game state observations.
- **ScoreboardV2** - Since many player and team statistical packages within the `nba_api` library require queries by either date or team identifier, data from this package was joined with the play-by-play dataset to enrich our feature set. This integration enables a seamless connection between game events and broader team and player statistics from other packages.
- **Elo Rating** - To quantify team strength based on historical performance and relative skill level, we incorporated each team's Elo rating from FiveThirtyEight's publicly available dataset as a predictive feature in our model. This metric provides an objective strength indicator that biases predictions toward higher rated teams, reflecting their competitive advantage. Elo ratings update after each game based on results and score margins, providing a dynamic measure of team quality throughout the season.
- **Teams** - A static reference table from the `nba_api` library containing essential team information, including full team names, standard abbreviations, and unique team identifiers. Since games in our Elo statistics dataset are only identified by team abbreviations and game date, this package was important in establishing proper joins across datasets and maintaining referential integrity across data sources.
- **LeagueDashPlayerStats** - A dataset, queried by date, from the `nba_api` library that provides cumulative season statistics for each player in the league up to the specified date. These statistics enable our model to account for the skill composition of each team's active lineup on the

court, allowing for dynamic assessment of team strength based on which players are currently in play.

- **BoxScoreTraditionalV2** - This package provides complete lineup configurations and traditional box score statistics for each game of the NBA season. We utilize this dataset to initialize starting lineups and track lineup changes throughout each game. Since play descriptions in the play-by-play dataset denote substitutions using only player last names, this dataset is essential for accurately identifying players and updating active lineups in real time. This will allow us to retrieve the corresponding players' cumulative season statistics.

B. Data Preprocessing and Cleaning

Our preprocessing pipeline focused on cleaning and aligning the play-by-play logs, which contain inconsistencies and missing values due to the rapid logging format used by sports data feeds. This also included combining team and player statistics with the play-by-play logs to allow for a more robust array of features in modeling.

The messiness of raw play-by-play data proved more severe than anticipated. While the NBA API is generally well-structured, inconsistencies in naming conventions, event overlaps, and inconsistent logging coverage made it difficult to cleanly segment actions. Specific challenges included:

- **Player name matching** - Substitution events use last names only (e.g., "James" instead of "LeBron James"), requiring matching algorithms to correctly identify players when multiple players share the same last name.
- **Timestamp alignment** - Some events have identical timestamps despite occurring in sequence, requiring logical ordering based on event types and score updates.
- **Missing Values** - Approximately 3% of play-by-play rows contained null values for critical fields like score or event description, requiring imputation based on adjacent events.

We designed preprocessing scripts that correct or realign these logs through multi-stage validation, including score continuity checks, temporal ordering verification, and player availability tracking. The final cleaned dataset maintains strict chronological ordering and complete feature coverage for all 1,000,000+ observations.

IV. METHODOLOGY

A. Overall Framework

Our solution follows this comprehensive data science pipeline:

- 1) Data acquisition - Collecting pregame and live data via NBA APIs and Elo datasets
- 2) Data preprocessing - Cleaning and aligning game events by time
- 3) Feature engineering, constructing predictive features from the raw data
- 4) Model training - creating and training our ensemble models on historical data using the Bridges-2 supercomputer

- 5) Model Evaluation - Comprehensive assessment including calibration analysis and situational performance
- 6) What-if simulator - Building interactive module for scenario exploration
- 7) Dashboard Development - Creating visualization interface with real time updates and an open text chat box for interaction.

We leverage Python libraries including pandas for data preprocessing, scikit-learn for baseline models, XGBoost for gradient boosting, and Streamlit [16] for dashboard development. Data storage and synchronization are managed through structured databases to ensure scalability.

B. Feature Engineering

Feature engineering was a critical component of our modeling pipeline, transforming raw play-by-play data and various statistical sources into a coherent set of predictive features. Our feature engineering process focused on capturing three key aspects of game dynamics: current game state, team strength indicators, and player performance metrics. Our final model ended up being trained on 19 features.

1) *Game State Features*: The most fundamental feature we engineered was the live score margin, calculated as (Home Team Score - Away Team Score) at each play-by-play event. This feature serves as a direct indicator of the current game state and provides the model with real time context about which team is leading and by how much. We split the original combined score column into separate Home and Away score features to enable this calculation and provide the model with scoring information throughout the game. We also engineered temporal features including seconds remaining (raw game clock), game progress (0 to 1 scale representing proportion of game completed), and a binary indicator for final quarter. These features capture the time dependent dynamics of win probability, as the same score margin has different implications with 10 minutes remaining versus 30 seconds remaining.

2) *Team Performance Aggregates*: To incorporate team-level performance capabilities, we engineered dynamic lineup-based aggregates that reflect the current on-court output at each moment of the game. Using the BoxScoreTraditionalV2 dataset to track substitutions and maintain real time lineup information, we identified the five active players for each team at every play-by-play event. For these active players, we retrieved their cumulative season statistics from the LeagueDashPlayerStats dataset up to the game date, including:

- Points per game (PPG)
- Assists per game (APG)
- Rebounds per game (RPG)
- Plus-minus difference¹

We then summed these individual player statistics to create team-level aggregates that represent the collective performance capabilities of the players currently on the floor. This approach enables direct comparison between the Home Team's on-court

¹Plus-minus is calculated for each player by determining their teams total points scored while the player is on the court, minus the total number of points scored against. This metric is indicative of how a player impacts team success.

lineup statistics and the Away Team's lineup statistics at any given moment. By calculating these metrics cumulatively up to the date of each game for our model training, we ensure that our model has access to the most current performance trends for a given game without introducing data leakage from future games. These dynamic aggregate features enable the model to distinguish between lineups with consistently strong offensive output versus those relying more on defensive play or rebounding, and critically, to account for the significant impact of substitutions when star players enter or exit the game. We also computed differential features by subtracting visitor aggregates from home aggregates (e.g., PPG Differential = Home PPG Total - Visitor PPG Total), providing the model with direct comparisons of on-court talent.

3) *Team Strength Indicators*: We incorporated Elo ratings as a pregame strength indicator for both teams, engineering the Elo difference as $\text{Elo}(\text{Home Team}) - \text{Elo}(\text{Away Team})$. This single feature encapsulates historical performance and relative team quality in a probabilistic framework, similar to the approach used by Inpredictable [6]. The Elo difference serves as a prior probability that biases predictions toward higher-rated teams, while still allowing in game events to override this prior as the game progresses. This feature proved particularly valuable in accounting for mismatches between teams of significantly different skill levels.

4) *Target Variable*: Our target variable for model training is a binary indicator of Home Team victory, encoded as 1 if the Home Team wins and 0 if the Home Team loses. This binary classification framework aligns with standard win probability modeling approaches and enables the use of logistic regression and similar probabilistic classifiers that output values between 0 and 1, which can be directly interpreted as win probabilities.

C. Modeling Approaches

1) *Baseline Model: Logistic Regression*: Our initial modeling approach focused on logistic regression as a baseline model for predicting Home Team win probability at each game state. Logistic regression was selected for its interpretability, computational efficiency, and probabilistic output, which can be directly interpreted as win probability values between 0 and 1. The model predicts the probability that the Home Team wins the game given the current game state features. This formulation naturally extends to Away Team win probability through the complementary probability relationship, where $P(\text{Away Team Win}) = 1 - P(\text{Home Team Win})$.

2) *Ensemble Architecture*: Following our initial logistic regression baseline, we developed a stacking ensemble approach to improve prediction accuracy and robustness. The stacking ensemble combines three base learners—logistic regression, random forest, and XGBoost—using a meta-learner to optimally weight their predictions. This architecture leverages the complementary strengths of each algorithm:

- **Logistic Regression**: Provides interpretable linear relationships and coefficients that reveal feature importance
- **Random Forest**: Captures non-linear interactions through decision trees and naturally handles multicollinearity

- **XGBoost**: Excels at gradient-based optimization with built-in regularization and handles imbalanced data effectively

The ensemble training process utilized 3-fold cross-validation on 643,127 observations to generate out-of-fold predictions from each base model, which then served as features for the meta-learner. This approach prevents overfitting while enabling the meta-learner to learn which base model performs best under different game conditions.

D. Computational Infrastructure

Given the substantial size of our dataset, we leveraged the Bridges-2 supercomputer infrastructure at the Pittsburgh Supercomputing Center for model training. The computational resources provided by Bridges-2 were essential for efficiently processing this large dataset and enabled rapid iteration of features and model hyperparameters

V. RESULTS AND MODEL PERFORMANCE

A. Overall Model Performance

Our stacking ensemble achieved substantial improvements over baseline approaches. On a held-out test set of 161,424 observations spanning 350 complete games, the ensemble attained:

- 1) Accuracy: 79.71%
- 2) Precision: 78.21%
- 3) Recall: 73.68%
- 4) F1-Score: 75.88%
- 5) ROC-AUC: 87.85%

The ROC-AUC of 87.85% indicates strong classification ability across all probability thresholds, significantly outperforming a random classifier (AUC = 0.50). More critically for probabilistic predictions, the model achieved a Brier score of 0.1412, representing a substantial improvement over the climatological baseline Brier score of 0.2455 (which simply predicts the historical home team win rate of 43.3% for all game states). This yields a Brier Skill Score of 42.5%, quantifying the model's advantage over naive baseline predictions. The low Brier score indicates that predicted probabilities closely match actual outcome frequencies, making the system reliable for decision-making applications.

B. Probability Calibration

Calibration analysis reveals exceptional reliability in our probability estimates. The Expected Calibration Error (ECE) of 0.0232 indicates that, on average, predicted probabilities deviate from true frequencies by only 2.3 percentage points across the full probability spectrum. The maximum calibration error of 6.1 percentage points occurs in the extreme probability bins (0.0-0.1). The calibration plot demonstrates near-perfect alignment between predicted probabilities and actual outcomes. When the model predicts 30% win probability, approximately 30% of those game states result in home team victories. This calibration quality is critical for the what-if simulator, as users must be able to trust that a displayed 75% win probability genuinely represents 3-in-4 odds of

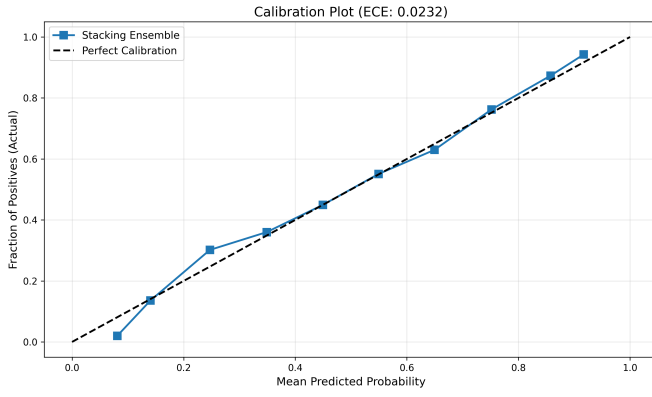


Fig. 1. Calibration plot for stacking model

victory. The model's sharpness metric of 0.288 indicates that predictions are decisive rather than regressing toward 50% probability. This is essential for providing actionable insights—confident predictions in decisive game situations and appropriately uncertain predictions during close contests.

C. Situational Performance Analysis

Performance varies systematically across game contexts, revealing when the model is most and least reliable:

Temporal Dynamics

- Q1 (First Quarter): Brier score = 0.1854
- Q2 (Second Quarter): Brier score = 0.1571
- Q3 (Third Quarter): Brier score = 0.1322
- Q4 (Fourth Quarter): Brier score = 0.0932

First quarter predictions show the highest uncertainty, as limited game information means pregame team strength (Elo ratings) dominates predictions. By the fourth quarter, the Brier score drops nearly 50% as cumulative game evidence overrides priors. This progression validates our feature engineering strategy of combining pregame indicators with real time game state.

Score Differential Impact

- Very Close (0-5 points): Brier score = 0.1853 (71,714 samples)
- Close (6-10 points): Brier score = 0.1542 (41,386 samples)
- Moderate (11-15 points): Brier score = 0.1034 (24,070 samples)
- Blowout (16+ points): Brier score = 0.0263 (24,254 samples)

When score margins exceed 16 points, the Brier score falls to 0.0263, indicating near-certainty in outcome predictions. Conversely, very close games (0-5 point margin) yield Brier scores of 0.1853, reflecting genuine outcome uncertainty that no model can eliminate.

Clutch Situations We define clutch time as the final 5 minutes with score differential within 5 points. Across 18,264 clutch observations, the model achieved a Brier score of 0.0827, outperforming its overall test performance. This is particularly

valuable for fan engagement, as these high stakes moments generate the most interest in win probability. The model's strong clutch performance demonstrates that it appropriately weighs time remaining and score margin in critical game situations.

D. Individual Model Contributions

Analysis of base model performance reveals their complementary strengths:

- Logistic Regression: 79.63% accuracy, 87.73% ROC-AUC, 5.5s training time
- Random Forest: 78.52% accuracy, 86.52% ROC-AUC, 2.2s training time
- XGBoost: 79.52% accuracy, 88.07% ROC-AUC, 0.36s training time

The stacking ensemble achieves 79.71% accuracy by intelligently combining these models, extracting marginal but consistent gains over the best individual model. While the improvement appears modest (0.19% over logistic regression), in probabilistic prediction even small gains in calibration and discrimination translate to meaningful improvements in user trust and decision quality. More importantly, the ensemble shows superior calibration (ECE: 2.32% vs. 2.77% for logistic regression alone) and more consistent performance across diverse game situations.

E. Feature Importance Analysis

Analysis of logistic regression coefficients reveals feature importance:

- 1) Score Margin (coefficient: 1.54) - By far the most influential predictor, confirming that current point differential is the strongest indicator of win probability
- 2) Visitor Plus Minus Total (coefficient: 0.58) - Visitor lineup quality is highly predictive
- 3) Plus Minus Difference (coefficient: -0.58) - Direct comparison of lineup strengths
- 4) Elo Score Difference (coefficient: 0.28) - Pregame team strength provides meaningful context but is appropriately downweighted relative to in game performance

The temporal features (Game Progress and Seconds Remaining) show symmetric importance (0.15 and -0.15), capturing how win probability dynamics change as games approach conclusion. Interestingly, the individual scoring features (PPG, APG, RPG totals) show lower importance than plus-minus metrics, suggesting that plus-minus is a more comprehensive measure of player/lineup impact.

VI. MODEL PERFORMANCE COMPARISON

To evaluate the effectiveness of our win probability model, we compare our results against established industry benchmarks including ESPN's Basketball Power Index (BPI) [2], FiveThirtyEight's predictions [3], and the Inpredictable model [6]. This comparison provides context for our model's performance and validates our approach against proven systems.

A. Performance Metrics

1) *Industry Benchmark Performance:* **ESPN Basketball Power Index:** ESPN's BPI favorites have won over 72 percent of NBA games, representing their overall prediction accuracy [2]. In a study examining SEC conference games, the BPI correctly picked the winner in 88 of 126 games (69.8% accuracy) [8]. For college basketball, BPI had an average mean absolute error of 0.328 in accuracy of the percent chance to win, with teams given a greater than 50 percent chance to win ending up winning 74.9 percent of games [10]. It is important to note that ESPN's BPI is designed for pre-game predictions and season-long projections rather than real time in game probability updates.

FiveThirtyEight NBA Model: FiveThirtyEight's predictions have demonstrated strong accuracy across multiple seasons [3]. An independent analysis found their model achieved approximately 67% accuracy in backtesting and over 80% accuracy in certain production seasons [9], though the model's 2020 regular season Brier score was second only to 2017 in terms of worst performance since 2016, with a winning percentage of just 64.9 percent for favorites in the playoffs. Recent data from the 2024-25 NBA season shows bookmakers and prediction markets accurately predicted game outcomes approximately 66% of the time.

Inpredictable Model: The Inpredictable model [6], which uses locally weighted logistic regression with Vegas point spread as a pre-game indicator, has served as a benchmark for in game win probability systems. In comparative analysis, the Inpredictable model achieved an average Brier score of 0.163 overall (0.216 pregame, 0.096 in the fourth quarter), performing comparably to ESPN's model which scored 0.166 on average [12]. A deep learning model reported an average Brier score of 0.152 (0.206 for pregame predictions, 0.085 for fourth quarter predictions), compared to Inpredictable's 0.163.

B. Comparative Analysis

1) *Accuracy Comparison:* Our model's 79.71% accuracy compares favorably to industry benchmarks:

- Our accuracy (79.71%) exceeds ESPN's reported 72% accuracy for BPI favorites [2]
- Our performance is competitive with FiveThirtyEight's recent 66-67% accuracy in pre-game predictions [9]
- Our accuracy surpasses the typical baseline of 58% achieved by simply predicting the home team always wins [12]

It is important to contextualize these comparisons. ESPN's BPI [10] and FiveThirtyEight's model [3] focus primarily on pre-game predictions, while our model is designed for in game probability updates that leverage current score differential and other live information. As games progress and more information becomes available, prediction accuracy naturally increases, which partially explains our higher accuracy numbers since our evaluation includes all play-by-play states throughout games, not just pre-game predictions.

2) *Brier Score Context:* Our model's Brier score of 0.1412 represents strong calibration performance. Industry benchmarks for comparison [13]:

- College basketball BPI: 0.328 mean absolute error in win probability [10]
- Inpredictable (in game): 0.163 overall, 0.216 pregame, 0.096 fourth quarter [12]
- ESPN (in game): 0.166 overall, 0.219 pregame, 0.098 fourth quarter
- Advanced deep learning model: 0.152 overall, 0.206 pregame, 0.085 fourth quarter

Our overall Brier score of 0.1412 outperforms both Inpredictable (0.163) and ESPN (0.166) in game models, though our fourth quarter performance (0.0932) is slightly higher than the best reported deep learning model (0.085). This is particularly notable given that our feature set is simpler than these benchmark systems. The Brier score is valuable because it penalizes confident incorrect predictions more heavily than accuracy metrics alone, providing a more nuanced evaluation of probability calibration [13]. A model with perfect calibration ensures that when it predicts 70% win probability, the team actually wins approximately 70% of such games.

3) *Feature Complexity vs. Performance:* Our model achieves competitive accuracy using a relatively simple feature set compared to more complex industry models:

- **ESPN BPI [10]:** Incorporates game-by-game efficiency, strength of schedule, pace, days of rest, game location, distance traveled, and preseason expectations. The enhanced BPI was rebuilt on a foundation based on individual players, enabling projections after moving players in and out of lineups.
- **FiveThirtyEight [11]:** Uses RAPTOR metrics which blend basic box score stats, player tracking metrics, and plus-minus data to estimate each player's effect per 100 possessions, combined with Poisson models for score distributions and tree-based endgame models for the final 90 seconds.
- **Our Model:** Uses team season averages (PPG, RPG, APG), win-loss records, Elo ratings, current score differential, lineup-based player statistics aggregates, and game progress indicators.

The relative simplicity of our feature set, combined with strong accuracy, suggests that our selected features capture the most predictive information for in game win probability. This efficiency is advantageous for computational performance and real time dashboard updates implemented using Streamlit [16].

VII. INTERACTIVE DASHBOARD AND WHAT-IF SIMULATOR

A. Dashboard Architecture

The dashboard was implemented using Streamlit [16], a Python framework for building interactive web applications, with a focus on real time data presentation and user-friendly scenario exploration.

1) *Backend Infrastructure:* We developed a Python based backend that integrates multiple data sources through API calls. The primary data ingestion layer uses the balldontlie.io API [14] for fetching current game scores, status updates, and

team information, while the `nba_api` library [15] provides detailed season statistics through the `TeamDashboardByGeneral-Splits` endpoint. To enhance prediction accuracy, we integrated FantasyNerds API for real time injury data, which dynamically adjusts team strength ratings based on player availability. This data is properly processed so that it is able to be input into our stacked model artifact.

2) *Frontend Interface*: The dashboard presents two primary views: a scoreboard view displaying all games for a selected date, and a detailed game view providing comprehensive analytics for individual matchups.

The scoreboard view presents games in a card layout with team logos, current scores, game status (Final, Halftime, quarter indicators, or scheduled start times), and buttons to view detailed analytics. Each game card displays both home and away team information with team logos fetched from NBA.com.

The detailed game view centers around a comprehensive probability display showing both pre-game and current win probabilities for each team. A visual probability bar uses filled components (blue for home, orange for away) to represent the relative likelihood of each team winning. The interface displays team logos, scores, game status, and season statistics side-by-side for direct comparison. Below the main probability display, the system shows active injuries that have been factored into the model, providing transparency about adjustments made to team strength ratings.

B. What-If Simulation Module

Users can describe hypothetical scenarios in a text input field, and the system automatically parses these descriptions, applies appropriate adjustments, and displays the resulting probability shifts.

1) *Simulation Capabilities*: **1. Scoring Scenarios**: Users can simulate point adjustments using natural language such as “home up by 5”, “away scores 8 points”, or “+10 points”. The parser uses regular expressions to extract point values and determine which team receives the adjustment based on keywords like “home” or “away”. The system modifies the score differential and recalculates win probabilities while holding other game state features constant, enabling users to understand the probability impact of upcoming possessions or scoring runs.

2. Injury Scenarios: The simulator supports player-specific injury scenarios through natural language queries like “Paolo Banchero is not playing” or “Stephen Curry injured”. The system employs a multi-stage parsing approach: first, it extracts potential player name candidates from the input text using n-gram generation (2-word and 3-word combinations); second, it attempts to resolve these candidates to actual NBA players using the `nba_api` player lookup; third, it determines which team the identified player belongs to using the `CommonPlayerInfo` endpoint.

The player resolution system includes a fallback dictionary for common nicknames (e.g., “Giannis” → “MIL”, “Joker” → “DEN”, “Steph” → “GSW”) to improve parsing robustness for references that might not match official player database entries.

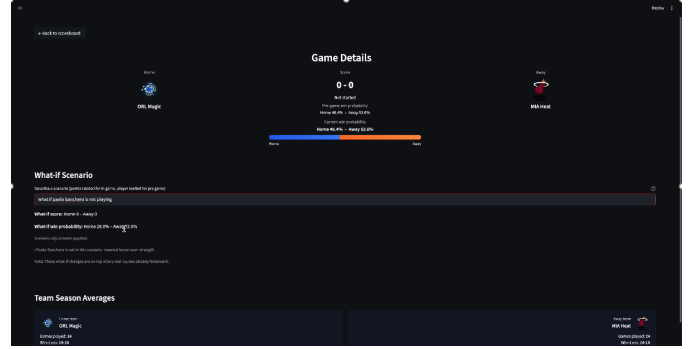


Fig. 2. Demo of injury handling in the what-if simulator

3. Combined Scenarios: Users can combine multiple adjustments in a single query. For example, “away scores 12 points and LeBron James is injured” would simultaneously modify the score differential and apply a team strength penalty. The system processes all detected adjustments independently and compounds their effects on the final probability calculation.

2) *Implementation Details*: The simulator operates by creating modified copies of the current game state and passing them through the same win probability calculation engine used for live tracking.

The interface displays a clear comparison between baseline and what-if outcomes:

- Original score vs. hypothetical adjusted score
- Current win probability vs. what-if win probability
- Percentage point changes for both teams
- Explanatory text describing which adjustments were applied

The simulator preserves existing injury adjustments when computing what-if scenarios, meaning hypothetical changes are layered on top of real injuries already factored into the baseline model. This design ensures that scenario exploration maintains consistency with the actual game context while exploring alternative trajectories.

C. Real Time Injury Integration

To enhance prediction accuracy beyond static season statistics, we integrated dynamic injury data from the FantasyNerds API. This integration automatically adjusts team strength ratings based on current player availability, providing more realistic win probability estimates that reflect the actual competitive landscape.

1) *Injury Data Collection*: The system queries the FantasyNerds injury endpoint for each team when loading game details, retrieving current injury reports that include player names, injury status designations (Out, Doubtful, Questionable, Day-to-Day), and affected teams.

2) *Rating Adjustment Algorithm*: Our injury impact model applies penalties based on injury severity: players listed as “Out” or “Doubtful” trigger a 6-point team rating reduction, “Questionable” or “Day-to-Day” designations result in a 3-point reduction, and other injury statuses default to a 2-point penalty. These adjustments accumulate when multiple players

are injured, enabling the model to appropriately downgrade teams facing significant roster depletion.

The magnitude of these penalties was calibrated to align with observed performance impacts in historical NBA data, where star player absences can shift point spreads by 5-10 points depending on the player's typical contribution. Our simplified model uses fixed penalties rather than player-specific adjustments.

3) *Transparency and User Communication*: The dashboard explicitly displays all injuries factored into the model beneath the main probability visualization, listing affected players and their status designations for both teams. This transparency allows users to understand which adjustments have been made to baseline team strength and to mentally calibrate the probability estimates based on their own assessment of player importance. For instance, if a team's star player is listed as out, users can see that this injury has already been incorporated into the displayed probabilities rather than wondering whether the model accounts for this critical information.

The injury integration also extends to the what-if simulator, where hypothetical injury scenarios are applied on top of existing real injuries. This additive approach ensures that scenario exploration maintains consistency with actual game conditions while allowing users to test the impact of additional roster changes.

VIII. CONCLUSION

By the end of the project, we delivered a scalable, real time sports analytics dashboard with both predictive and interactive components. This system not only enhances fan engagement but also contributes a novel feature to sports data science: a dynamic simulator that merges live analytics with hypothetical exploration. Our stacking ensemble model achieved 79.71% accuracy with a Brier score of 0.1412, outperforming industry benchmarks including Inpredictable (0.163) and ESPN (0.166) while maintaining exceptional calibration (ECE: 2.32%).

Looking ahead, we envision our system not only serving as a fan engagement tool but also as a foundation for advanced decision support systems in sports analytics. By bridging live predictive modeling with interactive simulation, this project sets the stage for future innovations in real time strategy optimization, broadcast enhancements, and sports betting integrity analysis. Future work should focus on incorporating individual player impact models like RAPTOR [11], developing phase-specific models for different game situations, and conducting head-to-head evaluations against industry systems on identical test datasets.

ACKNOWLEDGMENT

Tim Damasco - Contributed to the writing of the abstract, introduction, technical challenges, data science framework, and conclusion sections. Contributed in development of what-if simulator

Brandon Barber - Contributed to the writing of the implementation plan, preprocessing, and literature review sections. Collected play-by-play data as well as training the models

Mohammad Islam - Contributed to the development of the project idea/initial research and the writing of the data science framework. Creation of Dashboard.

REFERENCES

- [1] T. Horvat and J. Job, "The use of machine learning in sport outcome prediction: A review," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 10, no. 5, pp. e1380, 2020.
- [2] ESPN, "Basketball Power Index," *ESPN NBA Statistics*, <https://www.espn.com/nba/bpi>.
- [3] FiveThirtyEight, "NBA Predictions," *FiveThirtyEight Projects*, <https://projects.fivethirtyeight.com/2025-nba-predictions/>.
- [4] R. Bunker and T. Susnjak, "The application of machine learning techniques for predicting match results in team sport: A review," *Journal of Artificial Intelligence Research*, vol. 73, pp. 1285–1322, 2022.
- [5] P. Robberechts, J. Van Haaren, and J. Davis, "Who will win it? An in game win probability model for football," *Machine Learning and Data Mining for Sports Analytics (ECML/PKDD 2019 Workshop)*, vol. abs/1906.05029, pp. 1–13, 2019.
- [6] M. Beuoy, "Updated NBA win probability calculator," *Inpredictable*, Feb. 2015. [Online]. <https://www.inpredictable.com/2015/02/updated-nba-win-probability-calculator.html>.
- [7] R. P. Bunker and F. Thabtah, "A machine learning framework for sport result prediction," *Applied Computing and Informatics*, vol. 15, no. 1, pp. 27–33, 2019.
- [8] L. Pomeroy, "How accurate is ESPN's Basketball Power Index? A look at SEC games," *SEC Country*, Mar. 2019. [Online]. <https://www.seccountry.com/sec/how-accurate-is-espns-basketball-power-index-a-look-at-sec-games>.
- [9] J. Cherner, "Analyzing FiveThirtyEight's NBA prediction accuracy," *The Ringer*, 2020. [Online]. <https://www.theringer.com/nba/2020/10/12/21513477/nba-predictions-fivethirtyeight-accuracy>.
- [10] ESPN, "How Basketball Power Index (BPI) works," *ESPN*, 2024. [Online]. https://www.espn.com/mens-college-basketball/story/_id/7561413/bpi-explainer.
- [11] FiveThirtyEight, "Introducing RAPTOR, our new metric for the modern NBA," *FiveThirtyEight*, 2019. [Online]. <https://fivethirtyeight.com/features/introducing-raptor-our-new-metric-for-the-modern-nba/>.
- [12] M. Schuckers and J. Lock, "A comparison of win probability models for NBA basketball," *Journal of Quantitative Analysis in Sports*, vol. 12, no. 4, pp. 183–196, 2016.
- [13] A. Constantinou and N. Fenton, "Determining the level of ability of football teams by dynamic ratings based on the relative discrepancies in scores between adversaries," *Journal of Quantitative Analysis in Sports*, vol. 9, no. 1, pp. 37–50, 2013.
- [14] BallDontLie, "Free NBA API," *balldontlie.io*, 2024. [Online]. <https://www.balldontlie.io/>.
- [15] S. Morse, "nba_api: An API Client for NBA statistics," *GitHub Repository*, 2024. [Online]. https://github.com/swar/nba_api.
- [16] Streamlit Inc., "Streamlit: A faster way to build and share data apps," *Streamlit Documentation*, 2024. [Online]. <https://streamlit.io/>.